

GDN Tree-Scan: Design and Optimization of Tree Verification for Recurrent-Hybrid Language Models

Zhiyuan Ma

zhiyuanma0520@gmail.com

Abstract—Tree speculative decoding for long-running coding agents must preserve coherent continuation across repeated generation and tool calls. We present GDN Tree-Scan, a Gated DeltaNet verifier that connects branch-local computation to accepted-path publication across recurrent, convolution, and attention state. Its implementation combines sequential scan and replay with draft-logit reuse, shared operand preparation, and attention parallelism. On Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both shared SWE-bench Verified Astropy tasks, the best recorded tree run achieves 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. In a separate ten-task deployment, all tree-served agent attempts terminated and produced nonempty patches, with no degeneration detected by the recorded trace checks. Separate arithmetic and continuation checks characterize the verifier’s numerical and state-management boundaries.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

GDN Tree-Scan turns a candidate token tree into a target-model forward and a native continuation state. Its design connects an ancestry-aware attention path, branch-local GDN computation, device probability arithmetic, and accepted-path publication across recurrent, convolution, and attention key-value (KV) storage. A shared descriptor carries the logical tree through each stage [6], [7]. This system boundary matters because a verifier can compute plausible candidate logits yet publish the wrong state for the next iteration.

The central execution choice is to verify branches in temporary state and replay only the selected path into the native running state. Scan and replay share a sequential node-update body, which makes their intended arithmetic relationship explicit. This choice exposes concrete optimization questions: how much branch state to cache, when to recompute ancestry,

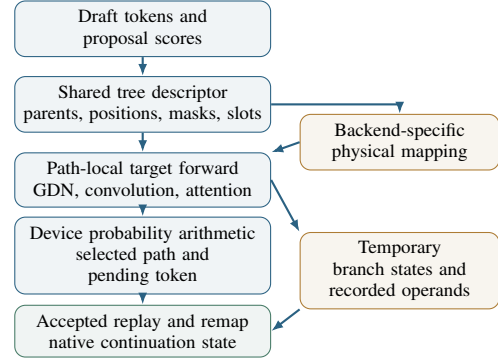


Fig. 1. The tree verifier as a serving pipeline. A shared descriptor connects candidate generation, ancestry-aware target computation, device probability arithmetic, and accepted-path publication. The attention backend determines the physical mapping; all state surfaces must describe the same selected prefix.

how to organize independent GPU work, and how to keep full-vocabulary probability arithmetic on the device. The attention layout and accepted-state mappings constrain those changes as much as the recurrent equation does.

The paper develops three implementation contributions. *First*, it presents the integrated verifier architecture and its continuation interface: candidates follow their ancestors during verification, and the selected materialized path becomes the next native state. *Second*, it explains the GPU scan/replay design, including its shared update, branch-state workspace, accepted-path publication, and memory–recomputation tradeoff. *Third*, it organizes the implemented optimizations around the work they remove and the numerical and backend constraints they must preserve: draft-logit reuse, device probability arithmetic, vectorized convolution, packed operands, attention layout/grouping, and scan/acceptance batching. We distinguish implemented candidates from the components active in the NVFP4 agent deployment and in the separate FP8 qualification route. Isolated gains require an ablation on the same agent workload.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, and a task can require many model requests. Performance therefore depends on useful completed work as well as token production. Our workload evaluation uses explicitly identified Astropy tasks from SWE-bench Verified [8]. We report the model, serving route, task cohort, and measurement definition together; a decode-rate change is not a task-completion speedup. Separate same-input arithmetic and continuation checks support

the implementation, without substituting for agent-workload measurements.

Tree verification and compact accepted-state reconstruction have direct prior art. Section II positions our implementation against those mechanisms; we do not claim the first hybrid-tree system or a new GDN recurrence. The architecture and kernel design are developed in Sections III–IV, and Section V separates optimization mechanisms from their measured scope. Measurement provenance is documented in Appendix A; superseded results and local-document timing numbers are excluded from the performance evaluation.

II. BACKGROUND AND RELATED WORK

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [9]–[11]. Our study keeps these algorithmic questions separate from whether a deployed kernel and cache return the intended probabilities.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [12], [13]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [14], [15]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. Our agent deployment uses patched FlashAttention-2. The recorded Cat10 numerical-qualification jobs use a separately patched Triton unified-attention path selected as `TREE_ATTN`. These are run-specific configurations, not a distinction imposed by FP8 versus NVFP4; backend-specific work partitioning remains part of the execution boundary [6], [16].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [17]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [18], [19]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It stores compact update factors and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-27B on GB10 and online replay of agent sessions [20]. Its state and serving mechanisms directly overlap the problem

TABLE I
DIRECT MECHANISM-COMPARISON SCOPE. THE AUTHORS’
IMPLEMENTATIONS OF THE THREE EXTERNAL METHODS LISTED HERE
WERE NOT EXECUTED IN THIS STUDY.

Method	Overlapping mechanism	Boundary for this study
Bole [20]	Parallel verifier; factorized commit	Different stack and precision; no matched run
TreeWY [21]	Tree solve; accepted-state reconstruction	Different models and operating regime; no matched run
ReplaySSM [22]	Input caching; deferred state work	Alternative state policy; no matched run
GDN Tree-Scan	Scan/replay; accepted-path remapping	Specific numerical and lifecycle witnesses; scoped cost evidence

studied here. The cost of our replay implementation is not a lower bound on alternatives.

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the source correction. Tree depth d gives $G^{d+1} = 0$, so $U = \sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [20]. Our path preserves sequential updates and replays accepted operands; Section V-E explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. Its vLLM evaluation uses Qwen3.5-35B and 397B on B200, greedy verification, and disabled prefix caching. It explicitly permits finite-precision disagreement with the baseline and reports that wider trees improve acceptance without yet winning throughput [21]. Consequently, our local numerical comparisons neither establish that TreeWY is incorrect nor replace its serving comparison.

ReplaySSM caches recurrent inputs and changes when state is materialized; its implementation RFC includes GDN and Mamba2 [22]. Reconstruction and replay are not claimed as new concepts here. Table I states the comparison boundary. Published speed ratios from different checkpoints, precision formats, workloads, and software builds are not combined into a performance ranking.

III. TREE-VERIFIER ARCHITECTURE

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$S(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(S(m), b)$, not a claim that $S(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 2 illustrates the three state surfaces documented by the implementation audit [6], [7].

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. A descriptor shared by verification and commitment

Figure 1 shows the served path. Native multi-token prediction (MTP) supplies the principal draft chain; selected variants add runner-up candidates. The descriptor maps each candidate through the attention mask, recurrent scan, sampler, and selected-path publication. The agent deployment uses Hydra27: 27 active draft nodes excluding the root, packed into 31 physical draft slots plus the root (32 verification rows). Its five-depth MTP head contributes 15 nodes; suffix continuation contributes a six-token principal tail and four- and two-token rescue branches. The maximum draft depth is eleven; four post-root MTP forwards construct the head. Four masked physical slots are padding, not extra logical candidates. Suffix candidates come from the configured Arctic suffix predictor. The separate Cat10 descriptor used in bounded continuation qualification contains the root and nine MTP draft nodes, with maximum draft-path depth five [6].

The patched `TREE_ATTN` qualification route uses flat candidate slots and the descriptor’s ancestry mask. Its cache-write addresses, attention-read addresses, and accepted-path remapping share that flat node convention. Recurrent nodes obtain their parent state from the pre-tree state or node-local scratch. After device selection, accepted-path replay and remapping establish the continuation state [6]. Cache checkpoints and restored rows must obey the same contract; a

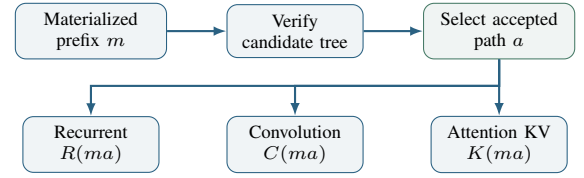


Fig. 2. The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled correction or bonus token can remain pending until the next forward.

Algorithm 1 Logical verifier and continuation boundary

Require: Reference-aligned state boundary, candidate descriptor T

- 1: Map logical candidates to physical slots and positions.
- 2: Verify using path-local convolution, GDN, and attention.
- 3: Apply the configured sampler to matching target/proposal rows.
- 4: Obtain the accepted path and any pending correction token.
- 5: Publish recurrent and convolution state for the materialized path.
- 6: Map its attention KV entries to the native continuation layout.
- 7: Map the selected hidden-state row to the next drafter input.
- 8: Preserve the native pending-token and position convention.
- 9: **return** Continuation boundary and emitted-token accounting

correct cold first request does not establish correctness after a checkpoint hit or page reuse [7].

IV. GPU SCAN AND ACCEPTED-PATH REPLAY

A. Verification workspace and native state lifetime

The branch-local scan evaluates nodes in a topological order. A root candidate starts from the pre-tree state; another node starts from its parent’s post-state. Packed adjacency alone supplies neither relationship. The cached implementation keeps node post-states in a tile-local table while writing the candidate outputs used by later layers. A separate activation ring records the operands needed to replay the chosen path after acceptance [6].

This workspace is speculative. The stateless serving route initializes it from request-owned native running rows and publishes the accepted state back to those rows. Recurrent and convolution publication must agree, since a later continuation or native cache operation can read both surfaces. Speculative scratch is retired by the selected lifecycle; no persistent leaf selector is needed to locate the next request state [7]. This is a state-management design, not a claim that speculative workspace consumes no memory. Fresh validation covers the

Algorithm 2 Logical scan and selected-path publication; compiled loops may include masked padding.

Require: Pre-tree state R_0 , descriptor T , rounded node operands x_i

```

1: for node  $i$  in topological order do
2:    $R_p \leftarrow R_0$  if  $i$  has no candidate parent; otherwise  $R_p \leftarrow H[\pi(i)]$ 
3:    $(H[i], o_i) \leftarrow \text{NodeStep}(R_p, x_i)$ 
4: end for
5: Finish target forward and select accepted materialized path  $A$ 
6:  $R \leftarrow R_0$ 
7: for node  $i$  in  $A$ , in path order do
8:    $(R, \_) \leftarrow \text{NodeStep}(R, x_i)$ 
9: end for
10: Publish  $R$  and matching convolution history to native running rows
11: Remap accepted KV entries and retire speculative scratch
12: return Matching continuation state and any pending token

```

selected eager route with prefix caching disabled, not every restore/eviction path.

B. A shared update with explicit rounding boundaries

Scan and replay call the same node-update body. Replay consumes the accepted path’s recorded operands in order, so it does not reconstruct the final state from an unrelated rounded output representation. The source exposes choices for raw-gate evaluation, query/key normalization, beta rounding, output storage, and carried-state precision. These choices must be pinned to a particular reference: the native speculative-update kernel and repeated one-token decode are distinct numerical references [6].

Sharing source makes the intended arithmetic common; it does not by itself guarantee identical compiled instructions. Operand shapes, launch geometry, unrolling, and compiler scheduling can change reduction or contraction order. We therefore separate the real-arithmetic recurrence from comparisons of compiled outputs and committed states. Section VII-B evaluates these quantities independently on the same fresh operands, and Section VII-D checks whether the next invocation consumes the state actually published.

C. Caching, replay, and recomputation

Let N_p be the padded node span, B_v the value-tile width, and d_k the key dimension. A cached scan’s fp32 node-state table has logical footprint

$$M_H = 4N_p B_v d_k \text{ bytes per program.} \quad (4)$$

For W warps, $N_p B_v d_k / (32W)$ is a useful estimate of cached fp32 values per lane, before live operands and compiler allocation. This is a resource model, not a measurement of allocated registers or spills. Padding can increase workspace even when the additional rows do not represent useful candidates. Narrower

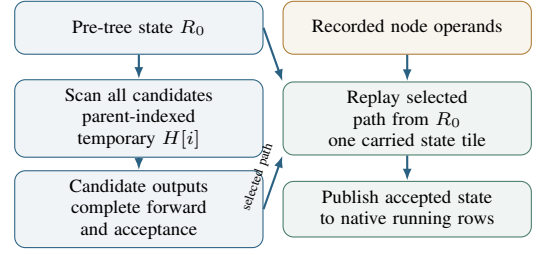


Fig. 3. Different state lifetimes within one verification step. The cached scan uses parent-indexed temporary states to produce candidate outputs. Accepted-path replay starts again from the pre-tree state and publishes only the chosen continuation. Compact reconstruction is a separately evaluated alternative.

tiles reduce the state table but create more programs and can alter the numerical execution layout.

Accepted-path replay instead carries one state tile and traverses a loop bounded by the configured path capacity, masking unused positions. Its selected-path semantics do not imply that arithmetic or latency scales only with the number of accepted nodes. Its carried-state footprint is $O(B_v d_k)$ in tree size, although recorded operands and the broader serving scratch still scale with candidates. Verification variants can similarly recompute a node’s ancestry or divide the tree into paths rather than retaining every node post-state. This exchanges stored state for repeated recurrence work; the useful choice depends on topology and GPU resource pressure [6]. None of these analytic footprints is a reported historical performance result.

Compact triangular-solve and finite-Neumann methods choose another point in this design space: they form correction factors and reconstruct the accepted state without replaying every accepted rank-one update. Our local comparison evaluates their verifier arithmetic and state reconstruction separately. Lower state-storage requirements or less replay work alone do not establish lower complete-step latency, and numerical agreement must be assessed at the actual output and state-storage boundaries [20], [21].

V. OPTIMIZATION MECHANISMS AND BACKEND CONSTRAINTS

A. Reuse draft logits and keep probability arithmetic on device

The native-MTP drafter supplies both a principal spine token and runner-up candidates from the same logits tensor. The single-logits path selects the spine token from that existing tensor, avoiding a second vocabulary projection inside the native greedy-sampling helper. Both the reuse path and its counterfactual OFF control obtain runner-up candidates from the first logits; the redundant work is the second projection for spine selection. Runner-up leaves use these scores without advancing an independent drafter state for each leaf. The descriptor keeps this proposal policy separate from target verification, allowing a different drafter to supply the same interface [6]. Reusing scores does not establish that a wider tree yields a net serving gain: its additional target rows still cost work.

The device multidraft committer keeps target/proposal probability arithmetic and residual selection on GPU tensors, avoiding a full-vocabulary probability transfer to the host. The bounded greedy qualification route still has Python request/node control loops and scalar readbacks; device-resident arithmetic is not a fully device-resident acceptance walk. The selected node sequence drives recurrent replay, convolution publication, KV remapping, and the next drafter’s hidden-row selection [6]. Greedy longest-prefix selection is not a substitute for a stochastic residual rule. The temperature-zero qualification checks establish only their stated greedy and continuation scope.

B. Prepare convolution rows and replay operands together

Tree convolution must gather a different causal window for each candidate path. The vectorized implementation builds static source indices over the prior history, current node inputs, and a zero row, then gathers the needed windows and state rows. Its tap computation uses explicit ordered additions and matching cast boundaries instead of replacing the original order with an unconstrained reduction. Preparing accepted-path row indices once per state group also avoids rebuilding the same mapping independently in each layer [6].

This vectorized convolution route and the eager packed-buffer preparation are active in the eager qualification implementation. The latter organizes recorded operands across layers and prepares shared metadata. Buffer packing and replay launch batching remain different mechanisms: the fresh greedy route follows canonical multidraft acceptance, whose unarmed replay branch still iterates over layers. The existence of stacked buffers therefore does not establish that all-layer replay was executed or that all replay launches were removed. Table II records this distinction; no isolated timing gain is assigned to either component.

C. Launch organization and independent acceptance work

The repository also implements candidates that fold independent GDN work into fewer launches and batch tree-acceptance work across requests. These attack launch overhead and underfilled device work, respectively. They retain per-request ancestry and output identities; a batching change must also preserve the intended normalization and cumulative-sum behavior. Attention grouping/padding candidates target reuse and execution geometry in the FA2 route. These are source-level optimization mechanisms, not automatically components of every tree configuration [6].

In the NVFP4 agent deployment, the FA2 GQA-pair kernel shares staged key/value tiles across a pair of query heads. Split-K partitions the context walk four ways; each partition writes a local maximum, denominator, and weighted-value sum. The combine kernel rescales those partials to the global maximum and merges them. This exposes more independent work without changing the logical tree visibility, but it changes floating-point accumulation order. It therefore requires its own numerical and serving evidence rather than inheriting qualification from the patched Triton path. The loaded task-run receipt confirms

this attention dispatch; its workload observations are reported without an isolated attention-speedup claim [6].

Table II separates this design inventory from the available evidence. The fixed-shape scan and acceptance candidates require compatible shape, backend, and launcher guards; they cannot be treated as interchangeable with the qualification route. Source availability and local-document timing do not establish their gains in an agent workload.

D. Publish the selected path in the backend’s address space

The current continuation path copies accepted attention KV entries from their verification slots into linear continuation slots. If accepted logical node a_j becomes continuation position j , the remapper moves its KV entry from the descriptor’s source slot for a_j to the request’s native destination slot for j . Recurrent replay, convolution publication, and the next drafter row use the same selected path. KV publication is state movement; it does not change the attention equation [6].

For the patched TREE_ATTN backend used in qualification, verification writes and attention reads retain flat candidate positions. The selected route enables accepted-path KV remapping and its sync-free helper, leaves slot reordering disabled, and uses synchronous scheduling. A layout permutation is valid only when both read and write mappings, ancestry visibility, and continuation publication use the same convention. Section VII-D reports the fresh incompatible-policy control and the checks of this selected flat-map implementation. The artifact binds effective post-boot settings to loaded source, so source availability alone is not treated as engagement.

E. Reconstruction, rounding, and accumulated disagreement

Verification outputs, committed state, and later model outputs are separate numerical boundaries. A first-order explanatory model is $e_{\ell+1} \approx J_\ell e_\ell + \delta_\ell$: later layers can amplify an existing residual while introducing additional error. This expression is not a fitted error bound. Small local errors, or matching current outputs, do not establish matching future logits. The fresh same-input and continuation studies below therefore evaluate verification arithmetic and state publication separately.

The numerical reference must also be named. The pinned native speculative-update and one-token decode paths differ in gate rounding and state-storage boundaries. Our scan and replay share a sequential update body, but their compiled agreement still requires GPU checks [6]. Thus a real-arithmetic reconstruction identity does not by itself establish deployment agreement; conversely, disagreement with our selected reference does not establish an algebraic defect. E7a and E7b isolate verification arithmetic, state commitment, and subsequent continuation before measuring any deployment benefit. E7a explicitly tests the native paths’ rounding conventions on fresh captured operands (Section VII-B); their different state-storage rules remain distinct reference contracts.

VI. EVALUATION SETUP

Performance is evaluated on coding-agent workloads, with the task subset and serving configuration attached to every

TABLE II
OPTIMIZATION-TO-EVIDENCE MAP. “IMPLEMENTED” DESCRIBES SOURCE AVAILABILITY; IT DOES NOT ASSERT A FRESH ISOLATED SPEEDUP OR ACTIVATION IN EVERY SERVING ROUTE.

Mechanism	Work or dependency addressed	Implementation boundary	Evidence / agent-workload boundary
Draft-logit reuse	Share one vocabulary projection for spine selection and runner-up candidates	Source-controlled ON/OFF variants in the qualification artifact	Same-input head/candidate checks; no agent-workload reuse ablation (Section VII-E)
Device probability arithmetic	Avoid full-vocabulary host transfer	Qualification route retains Python control and scalar readbacks	Bounded selected-path checks; no device/host agent-workload ablation
Vectorized tree convolution	Reuse static path-window indices and ordered tap operations	Active tree-only helper in qualification; enabled in task route	No fused/legacy agent-workload ablation
Packed eager buffers	Share operand storage and metadata preparation	Active in qualification; canonical greedy replay loops over layers	All-layer replay must not be inferred from buffer engagement
Scan/replay state policy	Temporary branch states; publish only accepted native state	Shared node update, activation ring, running-row lifecycle	Fresh arithmetic and continuation checks; state-policy performance ablation absent
Accepted-path KV remap	Publish selected continuation addresses	Patched Triton qualification route: flat slots, sync-free remap, no slot reorder	Incompatible-policy control and selected-route checks; backend-specific mapping
Attention grouping / split-K	Reuse query groups and divide attention work	Patched FA2 in the agent run; patched Triton in the qualification jobs	Active in the task case; no matched full-task ablation
Single-launch scan / batched acceptance	Reduce launch overhead / combine independent request work	Fixed32 single-launch GDN disabled in task case; other candidates retain guards	No composed optimization attribution on agent tasks

reported rate. The Qwen3.8-27B NVFP4 evidence uses SWE-bench Verified Astropy tasks on a single GB10/DGX Spark: a shared two-task comparison of three tree runs against SGLang EAGLE, plus a separate ten-task tree deployment [6], [8]. The best recorded shared-task rate supplies the headline; later runs and the larger deployment remain in the evaluation. These selected subsets are not the full benchmark. Every decode-rate result uses the pooled completed-request estimator in Eq. 5, excluding first-output latency. Recorded agent duration includes the agent loop, model service, tools, and repository operations; server startup and subsequent evaluator time are separate.

Numerical qualification is a separate experiment family on Qwen3.6-27B-FP8. It uses captured operands, internal-document prefixes, and explicitly labeled synthetic controls. The selected Cat10 continuation-qualification route uses patched Triton unified attention through `TREE_ATTN`, flat slots, accepted-path KV remapping, eager synchronous execution, and disabled prefix caching. These finite checks qualify that route only. They are not a SWE-bench run and do not automatically qualify a different NVFP4, graph, sampling, or attention-kernel configuration. Local-document throughput experiments are retained in the artifact for audit but provide no performance number in this paper.

VII. RESULTS

A. SWE-bench Verified coding-agent workload

We evaluate recorded deployments on Astropy instances from the test split of `princeton-nlp/SWE-bench_Verified`, using Qwen Code 0.19.4 to inspect repositories, edit files, execute tools, and produce patches. A shared two-task comparison covers three tree runs and one SGLang EAGLE run that each

produced nonempty patches on both tasks. A separate ten-task segment, Cqc10, recorded on 24 August 2026, supplies the latest complete single-configuration deployment case. The task/evaluation worker is x86-64; model serving runs on the GB10. These are full agent attempts. Table IV lists every task and saved evaluator outcome in Cqc10.

a) *Ten-task deployment*: The served model is the port’s `qwen3.8-27b-nvfp4-radixark` checkpoint with ModelOpt mixed quantization and `bfloat16` execution dtype. The route uses Hydra27 with 32 physical verification rows, full-vocabulary draft logits, draft-logit reuse, and a patched FA2 GQA-pair split-K4 attention kernel. Prefix caching and full/piecewise CUDA graphs are enabled. The engine permits one active sequence, with maximum context 131,072 tokens and a 4,096-token scheduler budget. Source commit `e7af6b595a8b` and the loaded extension hash are retained in the artifact. This graph/cache-enabled stochastic deployment is distinct from the Cat10 eager qualification below; passing that qualification does not prove this route’s full-model equivalence.

The proxy supplies temperature 0.6, top- p 0.95, top- k 20, minimum- p 0, and presence penalty 1.0; the engine seed is zero. The per-response output cap is 24,000 tokens, with a separate 20,000-token compaction cap and a 9,000-second agent timeout. Proxy auto-continuation is disabled. The agent runs in the task’s repository image on an internal network that permits the model proxy and blocks external egress; per-task records retain the verified network-policy fingerprint. Agent duration comes from the task runner’s elapsed-time field; it includes the agent loop and tool work, and excludes server startup and subsequent evaluation. We do not infer a completed-task speedup from model-only counters.

b) *Workload observations*: All ten attempts have evaluator reports: six resolved their tasks and four failed tests. Recorded

TABLE III

EVIDENCE SCOPE. ONLY NAMED CODING-AGENT WORKLOADS SUPPLY PERFORMANCE RESULTS. LOCAL NUMERICAL AND CONTINUATION TESTS SUPPORT MECHANISM CLAIMS; THEY DO NOT MEASURE AGENT PERFORMANCE.

ID	Evidence	Supported statement	Important boundary
P0	Configuration audit (Appendix A)	Sampling reconstruction; no new model measurements	Historical binary gaps retained
E7a	Same-input kernel study (Section VII-B)	Fresh synthetic controls, eight fresh pilot prefixes, and 31 of 32 confirmation prefixes passing the frozen provenance checks	Local realizations, not the authors' systems; frozen numerical/control criteria missed; no model-level qualification
E2/E7b	Stage-isolated and selected-route checks (Sections VII-C–VII-D)	One-prefix propagation diagnostics and bounded B1/B4 serving-contract evidence	No live conv/KV byte snapshots, full-model sequential-equivalence or general sampler-law proof
SWE	Qwen3.8 NVFP4 coding-agent records (Section VII-A)	Pooled decode observations from patch-producing tree/SGLang runs on shared tasks; separate ten-task deployment	Retrospective nonempty-patch filter; no draft-budget tuning sweep
E3	Conditional; not promoted	—	No composed optimization result

TABLE IV

ALL TEN TASKS IN THE COMPLETED CQC10 ASTROPY CASE. IDS HAVE PREFIX ASTROPY__ASTROPY-. TIMES ARE AGENT ELAPSED MINUTES, ROUNDED FROM SAVED RUNNER METADATA; EVALUATION IS SUBSEQUENT. THIS SELECTED DEVELOPMENT SEGMENT HAS ONE ATTEMPT PER LISTED TASK AND NO MATCHED NATIVE ARM.

Task suffix	Agent time (min)	Saved evaluation
13977	24.59	Failed tests
14096	22.99	Resolved
14182	7.23	Failed tests
14309	1.67	Resolved
14365	6.78	Failed tests
14369	27.70	Resolved
14508	41.24	Resolved
14539	18.57	Resolved
14598	25.79	Failed tests
14995	4.59	Resolved

agent durations sum to 181.15 minutes, with a range of 1.67–41.24 minutes. This is an observed task ledger, not a representative SWE-bench Verified success estimate. The sum is accumulated agent time, not elapsed campaign time including startup and evaluation.

Every task used tools and produced a nonempty patch. A source-bound replay of the recorded trace checks detects no degeneration in these ten runs and no malformed tool-call arguments. The degeneration heuristic jointly requires at most 200 visible characters, zero tool calls, and at least 2,000 thinking characters; none of the ten traces meets that conjunction. This screen can miss loops that continue narrating or using tools, so its zero detections establish neither universal degeneration freedom nor preserved task quality. The earlier campaign's detected degeneration remains an adverse observation in the lineage audit.

c) *Pooled decode comparison:* For completed request r , let n_r be its output-token count, ℓ_r its server-recorded end-to-end latency, and f_r its time to first output. With R completed requests and $N = \sum_r n_r$, we use

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)} = \frac{N - R}{\sum_r \ell_r - \sum_r f_r}. \quad (5)$$

Counts and durations are pooled before division. The numerator uses the conventional $n_r - 1$ output intervals; outputs arriving in one batch can have zero arrival gaps. The denominator excludes first-output latency and between-request agent/tool work. It is accumulated request time, which counts overlapping requests separately, not isolated GPU time or elapsed campaign time.

For Cqc10, $N = 248,342$, $R = 265$, $\sum_r \ell_r = 10,667.6211$ seconds, and $\sum_r f_r = 989.8239$ seconds yield 25.63 tokens/s. The 265 requests reconcile with 256 normal requests and nine successful internal compaction requests; all recorded completion reasons are stop. The ten pre/post metric brackets are contiguous. Generation-token and completed-request counters agree with their latency-histogram populations, with no counter reset or outstanding request at the selected boundaries.

The eligible SGLang EAGLE comparator, recorded on 24 August, serves the as-shipped Qwen3.8-27B NVFP4 checkpoint with FlashInfer, using speculative steps/top- k /draft-token settings of 3/1/4. It uses the same Qwen Code version, agent harness, requested sampling settings, and network policy, with a 24,000-token response cap, a 262,144-token context limit, and an 8,192-token prefill chunk. Its two completed Astropy tasks are 12907 (resolved; 7.10 agent minutes) and 13033 (failed tests; 26.96 agent minutes). Their counters give $N = 47,854$, $R = 45$, $\sum_r \ell_r = 2,013.6344$ seconds, and $\sum_r f_r = 235.9488$ seconds, yielding 26.89 tokens/s. Both brackets contain streaming requests only. All completed engine requests are included, including internal agent traffic.

d) *Performance eligibility:* After inspecting the archived runs, we restrict the shared-task rate comparison to run-pairs in which both tasks completed and produced nonempty patches. We apply this retrospective rule to both systems and retain patches that fail tests. All three tree pairs and one of the two recovered SGLang pairs meet the rule. The other SGLang pair is excluded: task 13033 produced an empty patch after a final thinking-only response at the configured output cap, so the test harness was not invoked. Its failed outcome and original records remain in the artifact. We exclude the entire pair from this performance comparison, rather than retaining only its successful task. The rates therefore characterize patch-

TABLE V

POOLED DECODE RATES FOR TREE VERIFICATION AND SGLANG EAGLE WITH QWEN3.8-27B NVFP4 ON GB10, USING EQ. 5. THE FIRST FOUR ROWS USE THE SAME TWO SWE-BENCH VERIFIED ASTROPY TASKS, 12907 AND 13033; EACH INCLUDED RUN PRODUCES NONEMPTY PATCHES ON BOTH TASKS; DECODER SETTINGS ARE SPECIFIED IN THE TEXT. CQC10 COVERS A SEPARATE TEN-TASK SUBSET. BOLD MARKS THE BEST RECORDED ELIGIBLE TREE RATE.

Deployment	Tasks	Requests	Tokens/s
SGLang EAGLE	2	45	26.89
Tree Sr12 (Aug 19)	2	41	29.09
Tree Cqc16 (Aug 19)	2	35	28.28
Tree Cqc15 (Aug 23)	2	53	27.27
Tree Cqc10 (Aug 24)	10	265	25.63

producing runs, not performance over all attempted tasks.

e) Best recorded shared-task result: We select task IDs 12907 and 13033 because both have completed SGLang records, then report all three recovered split-K tree runs on that pair. Sr12 and Cqc16 ran on 19 August; Cqc15 ran on 23 August. Their engagement receipts identify Hydra27 with 27 valid draft nodes, maximum draft depth 11, 32 physical verification rows, full-vocabulary drafting, and GQA-pair split-K4 on all sixteen full-attention layers, with no fallback. They share the loaded attention extension but differ in Python patch revisions. Each uses a 32,768-token response cap, compared with the eligible SGLang run’s 24,000. All 41 Sr12 responses and all 45 eligible SGLang responses are at most 20,000 tokens, below both configured caps; no response uses the differing output allowance. All Sr12 requests terminate with stop.

The best shared-task tree observation is Sr12: $N = 25,027$, $R = 41$, $\sum_r \ell_r = 989.2612$ seconds, and $\sum_r f_r = 130.4422$ seconds give **29.09 tokens/s**, 8.18% above the eligible SGLang rate under Eq. 5. Later observations are 28.28 for Cqc16 and 27.27 for Cqc15; Table V retains them alongside the best result. All selected task brackets pass the same population and idle-boundary checks. Every row on the shared pair has one resolved task and one failed task. All six selected tree attempts used tools and produced nonempty patches, with no detected degeneration or malformed tool-call arguments under the recorded checks. Every selected tree request finished with stop, without a recorded response-cap hit.

The tree headline is its best recorded rate on the shared pair, not the latest run or Sr12’s entire workload. Sr12 completed four tasks in total, two resolved and two failed; pooling all four gives 26.21 tokens/s. Cqc16’s subsequent task 13236 exhibited degeneration, and Cqc15’s subsequent task 13398 lacks a completed evaluation. Those adverse records remain in the artifact and outside this explicitly two-task comparison. The shared-task comparison evaluates the complete decoding systems, including their proposal topology, draft budget, and serving optimizations. These method-specific choices are part of the comparison. The records use the same requested sampling settings and agent harness. The performance table retains every eligible run-pair: three tree observations and one SGLang observation. This retrospective subset does not

establish performance over all attempts or each system’s optimum over draft lengths and candidate budgets. Repeated runs with a predeclared tuning protocol would measure that performance envelope; a component ablation would separately isolate the contribution of tree verification. The artifact retains the original metric brackets, source-bound configuration and behavior records, and reproducible pooled reductions.

f) Selection and comparison limits: Cqc10 contains the ten remaining tasks of an interrupted sixteen-task development campaign. Earlier segments changed source revisions and output caps and included degeneration, incomplete output, and budget-capped attempts. Source-bound audit records of those failures, retries, and missing evaluations remain in the artifact; the original records remain at their recorded source paths. We do not relabel the completed segment as the entire campaign, combine those segments into a single-configuration benchmark score, or drop unsuccessful attempts from a claimed benchmark denominator. The tasks were selected from one repository, and this deployment has one recorded boot without a matched ten-task native run. Separate targeted native probes change other serving settings and do not supply that comparator. Consequently, these records demonstrate an implemented verifier serving real coding-agent work, but establish no native-versus-tree task-speed or quality advantage. A current, matched task campaign is the remaining experiment for that claim.

g) Existing comparator experiments: Earlier SGLang EAGLE trials and native MTP controls remain in the repository. The earlier native campaign predates the NVFP4 deployment; later NVFP4 native probes cover targeted tasks with different serving settings. DSpark measurements use synthetic requests, and a later DFlash2 check establishes boot and response handling rather than agent-task performance. These records are distinguished from the pooled SWE comparison above; their superseded or non-agent rates are not additional performance results for the current verifier.

B. E7a: arithmetic on fresh operands and synthetic controls

The final confirmation study retains 31 provenance-passing inputs from 32 frozen prefixes and does not satisfy all original promotion criteria. Neither compact candidate is promoted. The pilot observations below explain the frozen choices; the subsequent confirmation and completed audit determine this final verdict.

E7a uses the rounded operands of one GDN layer step: bfloat16 q, k, v , bfloat16 raw gates, fp32 $A_{\log}$, bfloat16 gate bias, the fp32 pre-tree state, and the tree descriptor. Three references remain distinct: a float64 serial recurrence on the same rounded operands, the pinned native speculative-update kernel run per root-to-node chain, and the pinned one-token decode kernel run token by token. We report the revision’s fresh pilot and confirmation captures below, with a separately labeled fresh synthetic stress test. Preliminary remeasurements on archived captures and legacy kernels remain outside the manuscript’s quantitative results.

A fresh eight-prefix pilot ran on 22 September 2026 on the intended configuration: the audited Qwen3.6-27B FP8

checkpoint (read-only), the pinned vLLM image, B1 eager execution with the patched Triton `TREE_ATTN` backend, and the served ten-node caterpillar on the stateless tree route, and a documented one-hunk repair of the source patcher that made that route runnable at HEAD (a drafter constant emitted only under a fixed-tree mode; recorded with original and repaired hashes). Ninety-six candidate prefixes were cut from local repositories in three context strata and scored once by a native MTP reference server; eight pilot and 32 disjoint confirmation prefixes were frozen from native-reference behavior alone (top-1/top-2 margin and context strata) before any tree output was read. Each pilot prefix received one boot and one greedy request, and one one-shot capture of layer 62’s first tree step after the full prefill (two prefill forwards for the 4096-token prompts under the server’s logged 2048-token budget). A fail-closed provenance inspector (51 checks per capture: hashed prefix and prompt-token binding, payload written inside the first tree step, the per-request scheduler trace carrying the response identifier, per-step alignment of the accepted draft paths with the API-returned token identifiers, exact decoding of those identifiers to the response text, exact geometry and dtypes, non-trivial pre-state) passed on all eight (four prompts of 256 tokens, four of 4096), with three earlier boots retained as failed captures. The eight sets are independent frozen prefixes; the harness run of record is `experiments/out-20260922T013313Z-e7a-fresh-pilot-v2/`.

Mechanism A is this worktree’s production scan and accepted-path replay. Mechanisms B and C are our Triton reimplementations of the two published families on the same masked system $(I + G)U = R$: forward substitution (the WY/TreeWY family) and the finite Neumann series $\sum_{m=0}^d (-G)^m R$ (d correction powers plus the identity, hence $d + 1$ terms; the Bole family), each followed by the compact reconstruction $S_a = P_a S_0 + \sum_{j \leq a} (P_a / P_j) u_j k_j^\top$. Dense products run either in fp32 or in tf32 tensor-core precision. Every accepted prefix, including zero acceptance, is committed, first from each method’s own factors and then from identical oracle factors.

A fresh synthetic tiny-gate pass (`out-20260921T231030Z-e7a-tinygates-v3/`; chains of depth 11 and 5 and the caterpillar with cumulative log-decay down to -912 , so that the path decay P_i underflows to exactly zero in fp32) produced no non-finite value in any realization. With fp32 dense products, the diagnostic fp32-output-store outputs and committed fp32 states stayed within $0.9\text{--}5.8 \times 10^{-7}$ relative to the corresponding oracle maximum. Every implementation evaluates path-decay ratios as $\exp(\log P_i - \log P_j)$ with the exponent masked before exponentiation, never as a literal quotient of underflowed products. The same run shows that with vanishing gates the strict-ancestor coupling vanishes too, so even the truncated Neumann series is accurate there; the truncation control fails only when coupling is present.

The eight-prefix pilot determined the candidate precision modes and frozen comparison rules. Before reading confirmation outputs, we selected A, B, and C with fp32 dense products; fixed each numerical tolerance from the corresponding pilot maximum and a declared margin; specified zero units in the

last place for bitwise classes; and fixed the decision changes that would alter the claims. The resampling unit was the frozen prefix. Pilot-only passing observations and kernel timing diagnostics are retained in the artifact; they provide no agent-performance result. The confirmation results and completed audit below govern the reported numerical verdict.

The confirmation set (artifacts of record `experiments/out-20260922T051611Z-e7a-fresh-confirmation/`) contains 32 disjoint frozen prefixes, sixteen each with 256 and 4096 prompt tokens. One boot hung after autotuning and was retried after the driver’s timeout. One captured prefix, p087, failed the frozen provenance inspector: duplicate-token siblings made its path walk choose a different node from the served committer. That failure remains in the 32-prefix denominator. A revised path-consistency inspector passes the capture retrospectively; it does not replace the original verdict or add p087 to the 31-set primary comparison.

On the 31 provenance-passing sets, the production scan’s fp32-store output is bit-identical to the native speculative-update kernel’s, and the served bfloat16 output is byte-identical to the production scan’s. The maximum absolute compact-commit state errors are approximately 7.81×10^{-7} against the oracle and 1.91×10^{-6} against native; replay maxima are 1.34×10^{-6} and 2.38×10^{-7} , respectively (rounded). B and C’s bfloat16 outputs agree with native on at least 99.9772% of elements (the minimum is C on p005, with 14 of 61,440 elements differing). The served output is within 4.9×10^{-4} of the oracle and within one bfloat16 unit in the last place on significant elements. Sibling reordering and repeat replay remain bitwise invariant in their tested controls.

The frozen criteria are not satisfied. Native and production bfloat16 stores differ in one element on each of two sets, despite identical fp32 stores. One set exceeds the frozen absolute output tolerance by 3% (9.2×10^{-8} versus 8.9×10^{-8}). The frozen ordering $A \leq B = C$ for output error fails on three sets, and B/C compact-state error equality fails on p031 by 6.9×10^{-8} . Padding from 16 to 32 nodes changes output and correction-factor bits for B and C on all 31 sets. The associated kernel timing diagnostics also miss their frozen stability checks; their values remain audit-only.

A retrospective completeness audit found that the first checker omitted padding, compact-state ordering, and timing half-width checks. The immutable freeze also incorrectly described the pilot as padding invariant and timing compliant: the referenced fresh-pilot-v2 data already fail padding on all eight prefixes and the declared timing-stability rule. We retain the original freeze and reports and record the corrections separately in `p0/monitor/2026-09-22-review-14-frozen-audit.json`. This is an audit of a flawed frozen analysis, not a corrected preregistration. These observations characterize the captured arithmetic; they do not qualify deployment, establish model-level harmlessness, or promote either compact route.

C. Stage-isolated model diagnostics

We next isolated verifier substitution on one frozen pilot prefix, p072, at B1 with eager execution and the same pinned FP8 stack. Three separate boots ran the instrumented production route, a matched sham that computed B’s fp32 forward-substitution result without publishing it, and a verifier-only arm that published that result at layer 62. All three passed provenance checks. Across twelve contiguous, input-aligned steps with ten candidate rows each, the production and sham captures had identical logits, hidden states, residuals, and emitted token sequences. The thirteenth capture was partial at the response boundary and was excluded from the aligned comparison. This pair’s agreement is a control observation, not a universal cross-boot noise floor.

Against that sham, substitution first changed layer 62 and then layer 63; layers 0–61 remained identical. The largest absolute final-logit differences were 0.15625 at step 3 and 0.125 at step 7. No argmax changed among the 120 matched candidate rows, and all 32 emitted tokens were unchanged. Across thirteen verifier invocations, the minimum local bfloat16-output equality fraction was 99.9788%, with a maximum absolute difference of 6.10×10^{-5} . These observations demonstrate propagation in this implementation and prefix, without establishing model-level harmlessness or a defect inherent to either published mechanism. Raw captures and the independently checked reduction are retained under `experiments/out-20260922T063019Z-e7b-r13-p072-batch/`.

An independent review also found a publication defect in the diagnostic commit hook: on paths with more than one accepted draft, replacing only the accepted-column copy left the authoritative running state unchanged when the next forward read column zero. That defect concerns our substitution harness, not the compact algebra. After repair, three new boots ran production, a matched commit sham, and B’s commit-only substitution on p072 (`experiments/out-20260922T071945Z-e7b-r16-p072-commit-batch/`). Production and sham captures were bit-identical, including their 32 emitted tokens. All thirteen compact publications changed the authoritative state; all twelve captured next-forward pre-states consumed the preceding published bytes exactly. Across twelve input-aligned verifier calls, the compact arm changed final logits in two calls (maximum absolute difference 0.15625), without changing an argmax or the emitted token sequence. Only eleven calls had complete post-commit boundaries within the API output; the terminal partial call and extra capture are not additional fully emitted continuation intervals.

For a separate fixed-input check, we injected the compact arm’s first published layer-62 state into an offline float64 recurrence using the sham’s subsequent recorded operands and accepted paths, with fp32 state storage between steps. The initial maximum state difference, 2.38×10^{-7} , remained at that value over ten fully API-bound continuation intervals; maximum output differences ranged from 1.79×10^{-9} to 4.85×10^{-9} . The production-to-sham control stayed exactly zero. This once-injected, one-layer experiment differs from the

live arm’s repeated compact commits and does not exercise cross-layer feedback. No amplification was observed in its tested maximum-state-error measure and horizon. All thirteen captured steps of the production and compact boots also met the declared layer-level numerical fidelity checks, and all twelve pre-state links were bit-identical to the prior publication. None of these checks qualifies convolution history, attention KV, the sampler law, or the full serving route; those surfaces remain separate E2 requirements. The recorded environments for all six diagnostic boots omit the plain-route attention-KV remap flag, whose loaded source defaults to disabled. Their local arithmetic and matched-arm observations therefore do not establish correct branch-path attention continuation; the corrected serving route is qualified separately in Section VII-D.

D. Backend-specific continuation checks

Repairing attention continuation required checking both sides of the loaded backend’s address contract. A candidate policy enabled KV remapping, spine-first slot reordering, and the sync-free remap helper together. In the loaded `TREE_ATTN` backend, however, cache writes followed the permuted slot mapping while unified-attention reads used flat block-table positions and the original node mask. A source-derived address fixture showed that query node 3 read current-tree nodes `[0, 1, 4]` instead of `[0, 1, 3]`. On p072, the candidate policy and flat-map control had identical input hidden states and positions, but first differed at layer 3, the first full-attention layer. The final-logit maximum absolute difference was 6.125, with one changed argmax row. This is an incompatibility between these implementation settings, not a defect in the GDN recurrence or a published tree-verification method.

The selected policy keeps flat node slots, enables attention-KV remapping and its sync-free helper, and explicitly disables asynchronous scheduling. In a corrected B1 boot, the first forward matched the earlier diagnostic flat-map control from this revision across all 64 hidden-state/residual pairs and final logits. Later continuations differed after branch acceptance; that control had KV remapping disabled and is not a continuation oracle. Twelve captured layer-62 publications and eleven next-forward links verified that the authoritative state was read byte for byte. Actual draft tokens, parent topology, and full-vocabulary logits reconstructed all 31 post-prefill API tokens along their accepted-plus-bonus paths. The initial prefill token had API/ledger and returned-logprob evidence, but no captured full-vocabulary logits. The 32-token API response was the exact prefix of 34 structural outputs, with truncation only at its final boundary. A sealed event ledger independently reconciled these outputs and eleven complete physical-step intervals. This capture boot was not a warmed timing cell.

Helper tests separately checked active recurrent rows, convolution-history remainder protection, pending-token consumption, accepted-node KV copying, and four-request isolation, including multi-block KV layouts and corruption controls. Loaded-source guards and call order connect these helper contracts to the selected route; they do not substitute for

live convolution/KV byte snapshots or a full-model sequential oracle.

An actual four-request pilot used two 256-token and two 4096-token prefixes. All 53 captured layer-62 computations met the declared numerical classes, and all 49 available publication-to-next-input links matched byte for byte, including request-row compaction as other requests finished. Eight complete B4 forwards supplied 32 request blocks with full-logit, candidate, position, and published-path evidence, covering 85 structural output tokens; 83 fell within the returned API responses. The complete ledger reconciled all four 32-token responses and seven complete B4-to-B4 intervals. The client had omitted direct token-ID reporting: independent exhaustive inversion of the exact pinned tokenizer and context-sensitive API formatter recovered all 128 IDs uniquely over the model’s 248,320 output IDs, without consulting the ledger or draft IDs. Original strings and the derived-ID sidecar are retained, and subsequent clients request IDs directly. These finite checks support the selected eager, cache-off, temperature-zero route within the tested boundaries; they do not establish full-model sequential equivalence or qualify the NVFP4 agent deployment.

That pilot also exposed a distinction between greedy token choice and tree-node choice. Two sibling nodes proposed token 561 with equal overlap weights; the loaded sampler randomly selected a source child even at temperature zero. The published path [1, 3] validly emitted tokens [13, 561, 8057], whereas another legal sibling choice would continue along [1, 2, 4]. A gate requiring the first, last, or longest matching path would therefore reject a legal execution. The repaired check validates the actual published ancestry, each accepted token against its parent’s target argmax, the terminal bonus, and the absence of nonterminal premature stopping at a node with a matching child; API-budget clipping is allowed only at that request’s final output boundary. Original gate failures and their corrections are retained. This finite path check does not prove distribution preservation for stochastic sampling.

E. Same-input draft-logit qualification

A source-controlled check evaluates the reused and recomputed vocabulary heads on identical hidden inputs. Two untimed boots use eight internal-document excerpts, with 32 returned tokens per excerpt. Each arm completes 103 proposals and 515 paired comparisons of complete logits, argmax, ordered top-two candidates, and packed candidate trees. The checks also cover input and random-state preservation and watched-tensor mutation. All eight returned streams agree across these two qualification boots. This bounds the tested head/candidate invariants; it is not full-model equivalence or agent-task quality evidence. The separate local-document timing measurements are not reported as agent performance.

VIII. DISCUSSION AND REMAINING OPTIMIZATION EVIDENCE

The mechanism results constrain the implementation choices. Branch-local recurrence and selected-path publication are separate operations, so a local verifier match cannot establish

correct continuation. The fresh incompatible-slot-policy witness also shows why a numerical layout technique must respect the attention backend’s address convention. The corrected synchronous flat-map route passes the bounded checks reported above; live convolution/KV byte snapshots, graph execution, cache-hit behavior, stochastic deployment, and full-model sequential equivalence remain outside that coverage.

Sequential replay is a useful reference implementation, but the fresh compact-route comparison does not establish that it is universally more accurate or faster. The local realizations miss frozen numerical, padding, and timing criteria; no compact candidate is promoted. These are implementation-specific findings on the captured operands. They do not establish a defect in the published TreeWY or Bole systems, which were not benchmarked here.

For long-running agents, task completion and task wall time are the relevant application outcomes. More decoded tokens can reflect additional reasoning, a longer tool loop, or degeneration rather than more completed work. The pooled decode comparison is therefore accompanied by task IDs, outcomes, agent/tool policy, and terminal budgets. It describes the recorded tree and SGLang deployments on their evaluated subsets; it does not establish a task-completion speedup or isolate the effect of tree verification.

Optimization attribution also remains route-specific. A kernel comparison within one NVFP4 tree implementation cannot be multiplied by a reuse gain measured on local-document prefixes or transferred to a different model and attention backend. The numerical checks above are evidence about the exact tested computations, not a substitute for task-level validation. Appendix B records the remaining workload comparison.

IX. CONCLUSION

GDN Tree-Scan implements tree verification for a recurrent-hybrid model by connecting an ancestry-aware target forward to accepted-path state publication. A shared descriptor coordinates attention visibility, branch-local GDN computation, convolution history, device probability arithmetic, and the next native continuation. Sequential scan and accepted-path replay make the state lifetime and arithmetic choices explicit; cached state, recomputation, physical layout, and independent-work batching expose the main optimization tradeoffs.

The numerical studies characterize verification arithmetic and publication on the selected qualification route; neither local compact-state candidate passes all frozen promotion criteria. On two shared SWE-bench Verified Astropy tasks with Qwen3.8-27B NVFP4 on GB10, among runs producing nonempty patches on both tasks, the best recorded tree run attains 29.09 pooled tokens/s versus SGLang EAGLE’s 26.89, an 8.18% higher rate. Later tree runs on the same pair attain 28.28 and 27.27; the separate ten-task deployment attains 25.63. All ten attempts in that larger deployment terminated with nonempty patches and no degeneration detected by the recorded trace checks. The common rate definition compares the complete decoding systems; draft budgets, selection scope, and task outcomes are reported alongside the rates.

The contribution is a concrete verifier design and its implementation mechanisms, supported by bounded numerical checks and workload-specific observations. A broader performance claim requires additional tasks and repeated runs with a declared tuning protocol. The present results do not establish broader task-quality preservation, full-model sequential equivalence, or general superiority over SGLang or the published compact-state systems.

APPENDIX A MEASUREMENT AND PROVENANCE

A. Sampling provenance remains an explicit limitation

P0 source reconstruction found repeated target-logit constraints in earlier campaign patchers [23]. Repeated filtering and differing bonus paths mean that the requested historical sampling settings do not identify a single effective sampler. Historical loaded-extension and model-file identities also remain unresolved; current hashes cannot recover them. We preserve those records as audit evidence and exclude their superseded rates. Every reported workload result must identify its own model, route, sampler, and measurement support. Numerical qualification records remain separate from task-performance evidence.

The stored records remain audit evidence, but do not provide a controlled-temperature estimate of the final system’s advantage. Likewise, an available correctness test counts as a current passing result only when its execution record establishes the relevant route [6].

B. Tokens, events, physical steps, and service time

For event-level timing diagnostics, let a_e be accepted drafts and c_e be actual emitted tokens in request verification event e . For ordinary nonterminal events, $c_e = a_e + 1$ because of a correction/bonus token. End-of-sequence and truncation instead require the actual count. A physical GPU step can contain several request events. An event-level rate requires output counts and wall intervals from the same event population; component attribution also requires matching the timed operations to those events. These diagnostics are distinct from the completed-request pooled decode rate in Eq. 5.

An archived instrument did not supply that matching: its numerator summarizes global request events, whereas its wall intervals retain a pure-decode subset. Its component spans also use different supports. Changing units or substituting a different global token total cannot recover the missing matched observations [6]. The raw counters, retrospective audit, and reproducible accounting are preserved in the artifact; the derived rate proxies, component chart, and proxy break-even calculation are excluded here. The pooled estimator in Section VII-A must not be replaced by these mixed-support proxies. TTFT, TPOT, queueing, cache misses, and full-service throughput remain separate quantities.

The coding-agent records use selected Astropy subsets of SWE-bench Verified [8]. Task-success claims require raw

evaluator outcomes and an explicit denominator that retains incomplete attempts; neither these subsets nor internal-document prompts represent the full benchmark.

C. Four different correctness questions

Algebraic correctness asks whether the recurrence or its reformulation computes the same real-arithmetic function. *Numerical agreement* asks whether particular kernels, layouts, and precisions agree with a selected reference. *Sampler correctness* asks whether the acceptance/residual rule returns its supplied target distribution. *Serving correctness* asks whether these properties survive cache restoration, row reuse, batching, and the next iteration [2], [6], [17].

For example, for target probabilities p and proposal probabilities q , accepted mass is $\min(p, q)$ under ordinary rejection sampling. Sampling the remaining mass proportional to $(p - q)_+$ gives the target marginal. The argument is conditional on the actual proposal law and target probabilities; substituting stale logits or an incorrect state changes the distribution being sampled [1], [2]. The same distinction applies to the multidraft reference used here [3], [6].

If every iteration supplies the reference conditional probabilities and publishes an equivalent continuation boundary, an induction over emitted tokens gives the reference joint distribution. This is a conditional argument, not a proof that the implemented system satisfies its premises. A tolerance test, zero observed garble, or equal task outcomes does not discharge all those premises.

APPENDIX B VALIDATION SCOPE

Table VI separates completed mechanism qualification from the agent-workload study needed for an application-speedup claim. The original local-document timing campaigns remain immutable artifact records and are excluded from the paper’s performance results. New workload experiments must freeze the same checkpoint, agent harness, task cohort, sampling, tools, network policy, and budgets across comparators before execution.

E7a compares local reimplementations of published mechanisms, not the authors’ systems. Identical captured operands and chosen paths isolate solver error from commit error. High-precision recurrence and each pinned native GPU path serve distinct reference roles. Its fresh synthetic stress test, the verified eight-prefix pilot, and the 32-prefix confirmation set evaluated against the record frozen from that pilot are reported in Section VII-B, including all identified frozen-rule failures and the retrospective audit of omitted checks. Those sample counts belong to E7a. Neither compact candidate met the frozen local criteria, so neither is promoted to an integrated serving or performance claim. E7b instead reports the bounded, single-prefix verifier-only and commit-only diagnostics and the offline fixed-input continuation in Section VII-C. The matched input-aligned windows are observed controls, not a general forced-path facility: the served route refuses the older forced-spine diagnostic. No combined-candidate confirmation

TABLE VI
VALIDATION AND REMAINING WORKLOAD SCOPE. NUMERICAL CORRECTNESS EVIDENCE AND CODING-AGENT PERFORMANCE ANSWER SEPARATE QUESTIONS.

ID	Question	Minimum comparison	Claim enabled
P0	What actually ran?	Archived binary, flags, sampling and denominator reconciliation	Correct configuration labels
E7a	Do arithmetic realizations agree?	Same operands: sequential, triangular solve, finite-Neumann; output and commit isolated	Numerical characterization
E2/E7b	Do errors amplify or corrupt continuation?	Fixed-prefix stage substitutions; layer/final logits; state/KV/conv, sampler and cache fixtures	Scoped continuation and implementation agreement
E3	Do optimizations combine?	Compatible scan/accept-walk toggles on the same coding-agent tasks	Workload-specific optimization attribution
E4	How does memory affect service latency?	Deferred: identical arrival trace and cache-capacity controls	Serving and cache explanation
E5	Is broader task quality preserved?	Deferred: held-out tasks and powered quality design; exact sampler checks stay in E2	Broader quality evidence
SWE	Does the tree improve agent completion time?	Required: repeated matched native/tree tasks on one NVFP4 checkpoint; retain outcomes and budgets	Workload-specific task latency and success
E7c	How do Bole/TreeWY systems compare?	Deferred: feasible matched authors' implementations	Competitive system comparison

campaign or powered task-quality study has been completed. Broader promotion would require those separate verification, commitment, combined-route, and continuation checks; the present failures remain reportable numerical findings.

E2 combines existing analytic multidraft identities, multistep sampling tests, biased-selector controls, and accepted-path state fixtures [6] with the source-bound and actual-route checks in Section VII-D. Graph execution, prefix-cache hits, stochastic deployment, and a full-model sequential oracle remain outside this finite qualification. Source availability alone is not a passing execution record, and historical numerical or garble checks do not prove distribution preservation.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification," 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving mamba2 with delta rule," 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "Stree: Speculative tree decoding for hybrid state-space models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, "Lumo FlyWheel implementation and experiment archive," https://github.com/MaCoredroid/Lumo_FlyWheel/tree/984f613d7885d9bf7898866827552b68a3199d0c, 2026, revision evidence snapshot; individual campaigns retain their own run identities.
- [7] —, "Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native," https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html, 2026, author experiment archive; accessed 2026-09-21.
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [9] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable, robust, and hardware-aware speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>
- [10] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle-2: Faster inference of language models with dynamic draft trees," 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [11] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "Opt-tree: Speculative decoding with adaptive draft tree structure," 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [13] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [14] SGLang Team, "Hybrid Models Meet SGLang: More than Full Attention," <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [15] SGLang Project, "Qwen3-Next SGLang documentation," <https://lmsys.org/mintlify/app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [16] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [17] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing Linear Transformers with the Delta Rule over Sequence Length," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [18] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, "Specmamba: Accelerating mamba inference on fpga with speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [19] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, "Component-aware self-speculative decoding in hybrid language models," 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [20] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, "Bole: Efficient Tree Speculation for Hybrid-Attention Language Models," 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [21] S. M. Ghantasala, "TreeWY: Speculative Verification for Gated DeltaNet Hybrids," 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [22] Dao AI Lab and NVIDIA, "ReplaySSM: Cache SSM Inputs Instead of State," <https://github.com/vllm-project/vllm/issues/47572>, 2026, implementation RFC; accessed 2026-09-21.
- [23] Z. Ma, "Every Lever We Pulled: A Speculative-Decoding Speed Campaign in Full," https://macoredroid.github.io/Lumo_FlyWheel/ever-lever.html, 2026, author experiment archive; accessed 2026-09-21.